Laboratório Prolog · Aula 13

Matheus De Oliveira Mascarello · RA 24120740-2 · Turma A

Gerado em 19/10/2026, 22:04:39 · tempo registrado no cronômetro: 0:00

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) deve responder L = scheme (unico fato com funcional). paradigma(java, logico) deve dar false, porque o unico fato sobre java diz orientado_a_objetos.

Saída do ola.pl

```
?- linguagem(A).
linguagem(prolog)
linguagem(scheme)
linguagem(java)
?- paradigma(prolog,A).
paradigma(prolog,logico)
?- paradigma(A,funcional).
paradigma(scheme,funcional)
?- paradigma(java,logico).
false.
--- experimentos
?- linguagem(A).
linguagem(prolog)
linguagem(scheme)
linguagem(java)
?- paradigma(java,A).
paradigma(java,orientado_a_objetos)
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) tem uma variavel: pergunta "qual e o paradigma de prolog?" e o Prolog devolve os valores de P que tornam a consulta verdadeira (P = logico). paradigma(prolog, logico) nao tem variavel: pergunta "e verdade que prolog e logico?" e a resposta so pode ser verdadeiro ou falso.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que "o programa nao sabe isso" (nao consegue provar). O Prolog responde false quando nao encontra fato ou regra que prove a meta (hipotese do mundo fechado). Aqui coincide com a realidade, mas o false significa "nao provado a partir da base", nao uma afirmacao de que Java nao e logico.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) tera 7 respostas (lucia, marta, tereza, joana, bia, ana, dora). (mae(M, F), mulher(F)) tera 4 pares: lucia-tereza, marta-joana, tereza-bia, joana-ana.

Saída do familia.pl

```
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
mae(joana,jake)
?- mae(jake,joana).
false.
?- pai(maria,jake).
false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
mae(joana,jake)
mae(joana,ana)
?- pai(A,jake).
pai(jorge,jake)
?- mulher(A).
mulher(lucia)
mulher(marta)
mulher(tereza)
mulher(joana)
mulher(bia)
mulher(ana)
mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
mae(lucia,tereza),mulher(tereza)
mae(marta,joana),mulher(joana)
mae(tereza,bia),mulher(bia)
mae(joana,ana),mulher(ana)
--- experimentos
?- mae(joana,A).
mae(joana,jake)
mae(joana,ana)
?- pai(A,jake).
pai(jorge,jake)
?- mae(A,B),homem(B).
mae(lucia,vern),homem(vern)
mae(marta,paulo),homem(paulo)
mae(joana,jake),homem(jake)
mae(dora,leo),homem(leo)
?- homem(A),mulher(A).
false.
?- mae(joana,jake),mae(joana,ana).
mae(joana,jake),mae(joana,ana)
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

"Quais sao os pares (Mae, Filha) em que Mae e mae de Filha e Filha e mulher?", ou seja, todas as maes e suas filhas. Teve 4 respostas: lucia-tereza, marta-joana, tereza-bia e joana-ana.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

A hipotese do mundo fechado: tudo o que nao esta na base e nao pode ser provado e considerado falso. O problema e que, no mundo real, a base quase sempre e incompleta: o false pode ser so falta de informacao (ex.: um sistema que diz que um paciente nao tem alergia so porque ninguem cadastrou a alergia).

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

Apareceram na ordem lucia, marta, tereza, joana, bia, ana, dora, que e exatamente a ordem em que os fatos mulher/1 foram escritos no arquivo. O Prolog percorre as clausulas de cima para baixo, entao a ordem do codigo determina a ordem das respostas.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake): paulo (irmao de joana, mae de jake). primo(leo, X): jake e ana (paulo, pai de leo, e irmao de joana, mae deles).

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Falham: f(X, X) = f(a, b) (X nao pode ser a e b ao mesmo tempo), [A|B] = [] (lista vazia nao tem cabeca) e 2 + 3 = 5 (= nao calcula). f(X, X) = f(a, Y) unifica com X = a e Y = a.

Saídas de regras.pl e unificacao.pl

```
===== regras.pl =====
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
avo(marta,jake)
avo(vern,jake)
?- avo(osvaldo,A).
avo(osvaldo,joana)
avo(osvaldo,paulo)
avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
irmao(jake,ana)
?- tio(A,jake).
tio(paulo,jake)
?- primo(leo,A).
primo(leo,jake)
primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
ancestral(osvaldo,vern)
ancestral(osvaldo,tereza)
ancestral(osvaldo,joana)
ancestral(osvaldo,paulo)
```

```
ancestral(osvaldo,jake)
ancestral(osvaldo,ana)
ancestral(osvaldo,leo)
ancestral(osvaldo,bia)
?- ancestral(A,jake).
ancestral(joana,jake)
ancestral(jorge,jake)
ancestral(lucia,jake)
ancestral(marta,jake)
ancestral(osvaldo,jake)
ancestral(vern,jake)
?- ancestral(jake,A).
false.
--- experimentos
?- irmao(A,B).
irmao(vern,tereza)
irmao(tereza,vern)
irmao(joana,paulo)
irmao(paulo,joana)
irmao(jake,ana)
irmao(ana,jake)
?- tio(A,B).
tio(vern,bia)
tio(paulo,jake)
tio(paulo,ana)
?- ancestral(A,leo).
ancestral(dora,leo)
ancestral(paulo,leo)
ancestral(lucia,leo)
ancestral(marta,leo)
ancestral(osvaldo,leo)
ancestral(vern,leo)
?- primo(A,B).
primo(bia,joana)
primo(bia,paulo)
primo(jake,leo)
primo(ana,leo)
primo(joana,bia)
primo(paulo,bia)
primo(leo,jake)
primo(leo,ana)
[vern,tereza,joana,paulo,jake,ana,leo,bia]
--- regras.pl SEM o X \-= Y em irmao (pergunta 2.2)
?- irmao(jake,A).
irmao(jake,jake)
irmao(jake,ana)
?- primo(leo,A).
primo(leo,jake)
primo(leo,ana)
primo(leo,leo)
===== unificacao.pl =====
--- constantes e variaveis
```

```
a = a   unifica
a = b   NAO unifica
A = a   unifica   A = a
--- estruturas
f(A,b) = f(a,B)   unifica   A = a   B = b
f(A,A) = f(a,b)   NAO unifica
f(A,A) = f(a,B)   unifica   A = a   B = a
pai(A,jake) = pai(jorge,B)   unifica   A = jorge   B = jake
--- listas: [Cabeca|Cauda]
[A|B] = [1,2,3]   unifica   A = 1   B = [2,3]
[A|B] = []   NAO unifica
[A,B|C] = [a,b,c,d]   unifica   A = a   B = b   C = [c,d]
--- aritmetica NAO e unificacao
2+3 = 5   NAO unifica
--- experimentos
f(A,B) = f(1,2)   unifica   A = 1   B = 2
g(A,b) = g(a,A)   NAO unifica
[A,B|C] = [1,2,3,4,5]   unifica   A = 1   B = 2   C = [3,4,5]
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Porque a definicao tem dois casos, como toda recursao. Caso base: ancestral(X, Y) :- progenitor(X, Y) (pai ou mae ja e ancestral). Caso recursivo: ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y) (X e progenitor de alguem Z que e ancestral de Y). Sem o caso base a recursao nunca pararia; sem o recursivo so achariamos pais e maes.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, irmao(jake, _) passou a responder tambem irmao(jake, jake), e primo(leo, _) passou a incluir primo(leo, leo) (em irmao(A,B) todo mundo virou irmao de si mesmo). Isso acontece porque X e Y podem ser a mesma pessoa: jake tem a mesma mae e o mesmo pai que ele mesmo. A meta X \== Y e necessaria para excluir a propria pessoa.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Em f(X, X) = f(a, b), o primeiro argumento liga X = a; o segundo exigiria X = b, mas X ja vale a e a e b sao constantes diferentes, entao falha. Em f(X, X) = f(a, Y), X = a e depois o segundo argumento unifica X (= a) com Y, entao Y = a. Resultado: X = a e Y = a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Nao de forma direta. Em Java ou Scheme uma funcao tem entradas e uma saida fixas: avos(neto) devolve avos; para achar netos a partir do avo seria preciso escrever outra funcao (netos(avo)). Em Prolog avo/2 e uma relacao: qualquer argumento pode ser dado ou ser variavel, e a unificacao + backtracking procuram os valores que faltam. A mesma regra serve para as duas perguntas.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

concatena(X, Y, [1, 2, 3]) vai listar todas as formas de dividir a lista em duas partes: 4 respostas ([]+[1,2,3], [1]+[2,3], [1,2]+[3], [1,2,3]+[]).

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
membro(b,[a,b,c])
?- membro(z,[a,b,c]).
false.
?- membro(A,[a,b,c]).
membro(a,[a,b,c])
membro(b,[a,b,c])
membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
tamanho([a,b,c],3)
?- tamanho([],A).
tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
concatena([],[1,2,3],[1,2,3])
concatena([1],[2,3],[1,2,3])
concatena([1,2],[3],[1,2,3])
concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
inverte([1,2,3],[3,2,1])
--- experimentos
?- concatena(A,[b,c],[a,b,c]).
concatena([a],[b,c],[a,b,c])
?- inverte([a,b,c,d],A).
inverte([a,b,c,d],[d,c,b,a])
?- tamanho([x,y],A).
tamanho([x,y],2)
?- membro(A,[]).
false.
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: tamanho([], 0) (lista vazia tem tamanho 0). Passo recursivo: tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1. Para tamanho([a,b,c], N) fica pendente, a cada chamada, o calculo "N is M + 1": tamanho([b,c],M1) espera, depois tamanho([c],M2), depois tamanho([],M3) da M3 = 0; na volta faz M2 = 1, M1 = 2 e N = 3.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque com X e Y livres concatena funciona "ao contrario": gera todas as divisoes de [1,2,3] em prefixo + sufixo. Pela 1a clausula X = [] e Y = [1,2,3]; por backtracking a 2a clausula tira a cabeca 1 e repete com [2,3], e assim por diante, dando 4 divisoes (n+1 para lista de 3). O anexar do Scheme so calcula list1 + list2 -> resultado; nao consegue receber o resultado e descobrir as listas de entrada.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O backtracking (junto com a unificacao). A 1a clausula membro(X, [X|_]) da X = a; ao pedir outra resposta, o Prolog volta ao ponto de escolha e usa a 2a clausula, que chama membro na cauda [b,c], gerando b e depois c. A recursao + backtracking substituem o laco.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3 mostra X valendo a estrutura 2+3 (nao calcula). X is 2 + 3 mostra 5. fatorial(N, 120) vai dar erro de instanciacao, porque N > 0 e N1 is N - 1 precisam de N conhecido.

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- A=2+3.
2+3=2+3
?- A is 2+3.
5 is 2+3
?- 5--=2+3.
5--=2+3
?- 2+3=5.
false.
?- A is 7 mod 3.
1 is 7 mod 3
?- A is 7-/2.
3 is 7-/2
?- A is 2^10.
1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
fatorial(5,120)
?- fatorial(0,A).
fatorial(0,1)
?- fib(10,A).
fib(10,55)
?- soma([1,2,3,4],A).
soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
ERRO: instantiation_error
?- A is B+1.
ERRO: instantiation_error
```

```
--- experimentos
?- A is 5*4+2.
22 is 5*4+2
?- A=5*4+2.
5*4+2=5*4+2
?- fatorial(10,A).
fatorial(10,3628800)
?- fib(20,A).
fib(20,6765)
?- fatorial(A,120).
ERRO: instantiation_error
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= unifica, sem calcular: X = 2 + 3 deixa X = 2+3 (a estrutura), e 2 + 3 = 5 da false porque a estrutura +(2,3) nao e igual ao numero 5. is calcula o lado direito e unifica com o esquerdo: X is 2 + 3 da X = 5; X is 5*4+2 da 22. =:= compara valores numericos, avaliando os dois lados: 5 =:= 2 + 3 e verdadeiro (e exige os dois lados instanciados).

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

Porque is e as comparacoes (>, <) exigem que o lado direito / os operandos ja tenham valor. Em fatorial(N, 120), a meta N > 0 (e N1 is N - 1) roda com N livre e da instantiation_error; o Prolog nao "inverte" a conta. Ja concatena so usa unificacao de listas, que funciona com variaveis em qualquer posicao. A aritmetica e avaliacao de funcao (uma direcao), nao uma relacao.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Igual: mesma estrutura recursiva, com caso base fatorial(0, 1) / fact 0 = 1 e caso recursivo n * fatorial(n - 1). Diferente: em Haskell fact e uma funcao que devolve o valor; em Prolog o resultado e um argumento extra (F) e cada conta intermediaria precisa de uma variavel e de is (N1 is N - 1, F is N * F1). O Prolog tambem precisa do teste N > 0 para nao casar a 2a clausula com 0, e o Haskell usa casamento de padroes na ordem das equacoes.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) dara false (\+ casado(X) com X livre falha, pois existe alguem casado). maximo2(5, 3, M) dara duas respostas (5 e 3). maximo3(5, 3, 3) dara true (errado), porque a 1a clausula nao casa com M = 3 e o corte nem e executado.

Saída de controle.pl e mensagem de ancestral_ruim

```
===== controle.pl =====
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
tenta ana e ana
tenta ana e bia
```

```
tenta ana e caio
resposta: mesmo_gosto(ana,caio)
tenta bia e ana
tenta bia e bia
tenta bia e caio
tenta caio e ana
resposta: mesmo_gosto(caio,ana)
tenta caio e bia
tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(A).
solteiro1(osvaldo)
solteiro1(paulo)
solteiro1(jake)
?- solteiro2(A).
false.
--- C) corte
?- maximo1(5,3,A).
maximo1(5,3,5)
?- maximo2(5,3,A).
maximo2(5,3,5)
maximo2(5,3,3)
?- maximo3(5,3,A).
maximo3(5,3,5)
?- maximo1(5,3,3).
false.
?- maximo3(5,3,3).
maximo3(5,3,3)
--- experimentos
?- maximo3(8,3,A).
maximo3(8,3,8)
?- solteiro1(A).
solteiro1(osvaldo)
solteiro1(paulo)
solteiro1(jake)
?- maximo2(8,3,A).
maximo2(8,3,8)
maximo2(8,3,3)
===== laco_infinito.pl =====
ancestral_bom(osvaldo, jake)?
sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)
Fatal Error: local stack overflow (size: 16384 Kb, reached: 16383 Kb, environment
variable
used: LOCALSZ)
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores existem 24 coloracoes (PR, SP e MS formam um triangulo e usam as 3 cores: 3! = 6 formas, e o resto fica determinado ou com poucas opcoes). Com 2 cores nenhuma (false), porque o triangulo PR-SP-MS precisa de 3 cores.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog escolhe X = ana (1o fato amigo) e depois tenta Y em ordem. Em "tenta ana e ana" a meta que falhou foi X \== Y; ele voltou ao ultimo ponto de escolha, amigo(Y), e tentou Y = bia. Com ana e bia, gosta da cafe e cha, e gosta(bia, cafe) falha; volta de novo e tenta Y = caio: os dois gostam de cafe -> 1a resposta (ana, caio). Ao pedir mais, esgotam-se os Y para ana e ele volta a amigo(X): X = bia (nenhum par serve, bia e a unica que gosta de cha) e depois X = caio, que com Y = ana da a 2a resposta (caio, ana). caio-bia e caio-caio falham e a busca termina.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

\+ e negacao como falha. Em solteiro2 o \+ casado(X) roda com X ainda livre: casado(X) consegue ser provado (X = vern), entao \+ falha e a regra inteira da false, sem nunca chegar a homem(X). Em solteiro1, homem(X) primeiro liga X a uma pessoa e o \+ testa so aquela pessoa. Osvaldo aparece como solteiro por causa do mundo fechado: nao existe fato casado(osvaldo), entao "nao consigo provar que e casado" vira "e solteiro" (ser pai de vern nao tem nada a ver com a regra).

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte so atua quando e executado. Em maximo3(5, 3, 3), a cabeca da 1a clausula maximo3(X, Y, X) tenta unificar X = 5 com o 3o argumento 3, falha antes de chegar ao ! e o Prolog passa para a 2a clausula maximo3(_, Y, Y), que aceita Y = 3 sem teste nenhum -> true. Correcao: unificar o resultado depois do corte:

maximo3(X, Y, M) :- X >= Y, !, M = X.

maximo3(_, Y, Y).

Testei: maximo3(5, 3, 3) passa a dar false e maximo3(5, 3, M) da so M = 5.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

Em ancestral_ruim a primeira meta do corpo e ancestral_ruim(X, Z): para provar ancestral_ruim(osvaldo, jake), o Prolog chama ancestral_ruim(osvaldo, Z), que chama ancestral_ruim(osvaldo, Z2)... sem consumir nenhum fato (recursao a esquerda), ate estourar a pilha. Em ancestral_bom, antes da chamada recursiva vem progenitor(X, Z), que consome um fato e aproxima de Y, entao a recursao termina. Logicamente sao equivalentes, mas a busca em profundidade da esquerda para a direita faz a ordem das metas importar.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

3 cores: 24 coloracoes; 2 cores: nenhuma (false); 4 cores: 1728. E declarativo porque o programa so descreve o que e uma coloracao valida (cada estado tem uma cor da paleta e vizinhos tem cores diferentes), sem laco nem algoritmo de busca. Quem faz a busca (gerar e testar) e o mecanismo de backtracking do Prolog.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No funcional, anexar (Scheme) e uma funcao: recebe list1 e list2 e devolve a juncao, em uma unica direcao. No logico, concatena(A, B, C) e uma relacao entre tres listas: o mesmo codigo junta listas, descobre o prefixo/sufixo que falta ou gera todas as divisoes de uma lista, porque o Prolog busca uma prova por unificacao e backtracking.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

1) Controle da ordem de resolucao: em laco_infinito.pl, ancestral_ruim e logicamente igual a ancestral_bom, mas estoura a pilha so por causa da ordem das metas (recursao a esquerda). 2) Negacao como falha / mundo fechado: em controle.pl, solteiro2 da false so pela ordem das metas, e solteiro1 diz que osvaldo e solteiro apenas porque nao ha fato casado(osvaldo). (Tambem: aritmetica nao relacional, fatorial(N, 120) da instantiation_error.)

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas de regras e busca: sistemas especialistas, consultas dedutivas sobre bases de conhecimento (como a arvore da familia), quebra-cabecas e restricoes (mapa, SEND+MORE), analise de linguagem. Nao escolheria para sistemas com muita aritmetica, estado, interface, I/O e desempenho previsivel (ex.: um sistema web de vendas ou um app comercial), onde Java tem melhor ecossistema, tipagem, ferramentas e controle explicito do fluxo.

Exercícios opcionais

Opcionais: resultado dos testes

```
Exercicio 1
ok      irma(ana, jake)
ok      irma(tereza, vern)
ok      NAO irma(jake, ana)
ok      tia(tereza, joana)
ok      tia(joana, leo)
ok      NAO tia(joana, jake)
Exercicio 2
ok      ultimo([a,b,c], c)
ok      ultimo([7], 7)
ok      NAO ultimo([a,b,c], b)
Exercicio 3
ok      soma_quadrados([1,2,3], 14)
ok      soma_quadrados([], 0)
ok      pares([1,2,3,4,5,6], [2,4,6])
ok      pares([1,3], [])
```

```
Exercicio 4
ok      mdc(12, 18, 6)
ok      mdc(17, 5, 1)
ok      mdc(10, 0, 10)
ok      potencia(2, 10, 1024)
ok      potencia(5, 0, 1)
Exercicio 5
ok      caminho(curitiba, foz)
ok      caminho(joinville, cascavel)
ok      NAO caminho(foz, curitiba)
ok      NAO caminho(curitiba, paris)
Exercicio 6
ok      3 cores: 12 coloracoes
ok      2 cores: 2 coloracoes
ok      1 cor: nenhuma
Exercicio 7
ok      colega(ana, bruno)
ok      NAO colega(ana, carla)
ok      NAO colega(ana, ana)
ok      ganha_mais(ana, bruno)
ok      NAO ganha_mais(elisa, ana)
ok      total_depto(ti, 12500)
ok      total_depto(rh, 9000)
```

E1 Regras de família: código escrito

```
irma(X, Y) :- irmao(X, Y), mulher(X).
tia(X, Y) :- progenitor(P, Y), irma(X, P).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|T], X) :- ultimo(T, X).
```

E3 As funções da Aula 12: código escrito

```
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, S1), S is S1 + H * H.
pares([], []).
pares([H|T], [H|P]) :- H mod 2 -== 0, pares(T, P).
pares([H|T], P) :- H mod 2 =\= 0, pares(T, P).
```

E4 Aritmética: código escrito

```
mdc(A, 0, A).
mdc(A, B, D) :- B > 0, R is A mod B, mdc(B, R, D).
potencia(_, 0, 1).
potencia(B, E, R) :- E > 0, E1 is E - 1, potencia(B, E1, R1), R is B * R1.
```

E5 Grafo com ciclos: código escrito

```
caminho(A, B) :- caminho(A, B, [A]).
caminho(A, B, _) :- liga(A, B).
caminho(A, B, Visitados) :-
liga(A, C), \+ membro(C, Visitados),
caminho(C, B, [C|Visitados]).
```

E6 Mapa menor: código escrito

```
mapa_sul(Paleta, [RS, SC, PR]) :-
membro(RS, Paleta), membro(SC, Paleta), membro(PR, Paleta),
RS \-= SC, SC \-= PR.
```

E7 Prolog como banco de dados: código escrito

```
colega(X, Y) :- funcionario(X, D, _), funcionario(Y, D, _), X \-= Y.
ganha_mais(X, Y) :- funcionario(X, _, SX), funcionario(Y, _, SY), SX > SY.
total_depto(D, Total) :- findall(S, funcionario(_, D, S), L), soma(L, Total).
% Comparacao com SQL:
% colega ~ SELECT a.nome, b.nome FROM funcionario a JOIN funcionario b ON a.depto =
b.depto
WHERE a.nome -> b.nome;
% total_depto ~ SELECT SUM(salario) FROM funcionario WHERE depto = 'ti';
```